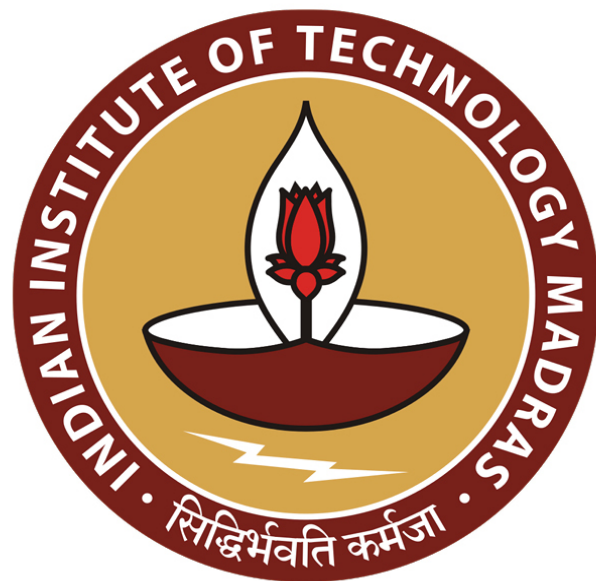




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science & Engineering
Indian Institute of Technology Madras
Principal Component Analysis Part - 1

Hello and welcome back. So, we will, in this video continue our discussion of this new algorithm that we are trying to develop for unsupervised learning, where we were trying to find a line that given a data set minimizes the sum of the lengths of the projections onto this line. But when we did that, we came up with a lot of questions. For example, we started with an error minimization problem and we post it as an equivalent maximization problem of a particular objective.

We want to understand that in detail. So, the second question is, we came up with a possible algorithm where we find iteratively multiple lines. And then we wanted to know, how many rounds should we run this procedure for? When should we stop this procedure, that was the second question.

And more fundamentally, we were trying to understand where is really the compression happening? And what are the representations that we are really learning in this problem? What we will do in this video is try to answer some of these questions. So that the algorithm that we are trying to develop kind of solidifies and then we have a solid algorithm in place.

(Refer Slide Time: 01:31)

$$D = \{x_1, x_2, \dots, x_n\} \quad x_i \in \mathbb{R}^d$$
$$w_1 = \underset{\|w\|^2 = 1}{\operatorname{argmax}_w} w^T C w \quad C$$
$$\begin{array}{lcl} x_1' & \rightarrow & x_1 - (x_1^T w_1) w_1 \\ x_2' & \rightarrow & x_2 - (x_2^T w_1) w_1 \\ \vdots & & \vdots \\ x_n' & \rightarrow & x_n - (x_n^T w_1) w_1 \end{array}$$

So, we will start today with where we were last time. So, we had a data set. Let us call this $\{x_1, x_2, \dots, x_n\}$ as usual, the data points are in d dimension and we have a vector w_1 that we are able to find for this data set, which is argmax over w , such that $\|w\|^2=1, w^T C w$. When you say argmax, it is the argument that maximizes the subject to function and which means specifically, that w that maximizes the subject to function, C here was just $\frac{1}{n} \sum_{i=1}^n (x_i x_i^T)$.

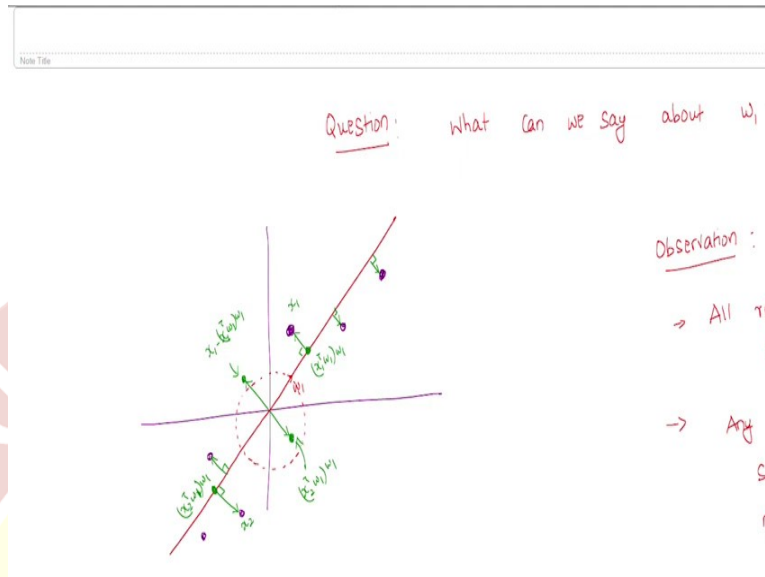
So, once you find this, w by the best line that kind of minimizes the error or maximizes this quantity, what we then said was, we would compute this new data set where the point x_1 becomes x'_1 , which is just $x_1 - (x_1^T w)w$

x'_2 would be $x_2 - (x_2^T w)w$ and so on and x'_n .

So, I mean, I am just calling it tilde it should be prime which is $x_n - (x_n^T w)w$. So, we have this as a new data set so, these set of points for our new dataset now. Remember, all these points are still in $\mathbb{R}^d$, just that they are the error vectors in d dimension. So, now what would our w_2 be our w_2 would be the argmax over w such $\|w\|^2=1, w^T C' w$, where C' is now the matrix $d \times d$ matrix that you compute out of these error vectors, which would simply be $\frac{1}{n} \sum_{i=1}^n (x'_i x'_i{}^T)$.

So, this would be the new matrix. So, now you have two lines, so, the first line was w_1 , which was the best line that minimize the error with respect to the data set now, you took the error vectors and formed the second line, which is w_2 .

(Refer Slide Time: 04:06)



Now, a natural question arises, we have two lines now, can we say anything about the relation between these two lines? So, natural question is the following. What can we say about w_1 and w_2 ? So, in particular, is there any specific relationship that exists between these two directions? So, let us look at it geometrically.

So, we have a bunch of data points. As usual, the data set let us say centered and now let us say this was our best line, it can fall on the line does not matter. But let us say to show it clearly, so let us say this was the best line w_1 . So, w_1 is somewhere here. To remember, so w_1 is unit length. So, it would be the direction that would point (along the) in the unit circle. Now, what are the error vectors?

If this is x_1 , this is our error vector, this point is $(x_1^T w_1) w_1$, which means this error vector, which is actually somewhere pointing this is the remaining piece, which is $x_1 - (x_1^T w_1) w_1$. So, it is pointing in this direction. Let us take one more error vector, which is perhaps this error vector, which is this vector, maybe this point is x_2 this point is $(x_2^T w_1) w_1$. So, this error vector is here. And this point is $(x_2^T w_1) w_1$ and so on. So, now, as you can observe that these error vectors all line up in the same line.

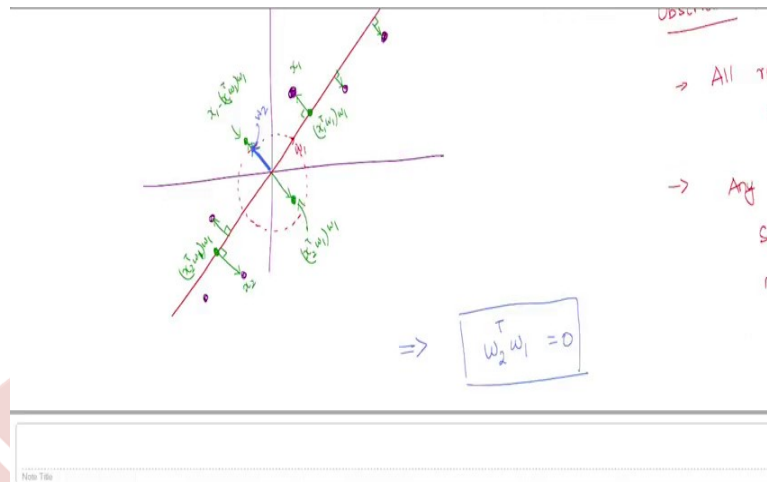
Now, why should that be the case that should simply be the case, because all these error vectors are perpendicular to the original line that we created that is because of the nature of projections here. So, the projections are all perpendicular. These are all perpendicular. And so, if we just create a data set which only has the error vectors that will be (per on the) that will be on the line perpendicular to the original line that we found so, in 2 dimensions.

In higher dimensions, it will be in some hyperplane, which is perpendicular to the original line that we found. But in 2 dimension there are only two perpendicular lines possible. So, it will be on a line. So, now, if we find w_2 which best fits these error vectors, and because of the fact that the error vectors are all lining up along the line, the best line would exactly be the line where they line up along.

So, because if you pick any other line, I am going to suffer some extra error, which I can avoid by picking w_2 as the line which is exactly the line where these error vectors are falling on, which is the line perpendicular to w_1 . So, which means the observation again, I did not prove this, but then the observation is that all errors or let us call them residues residues are orthogonal to w_1 .

And this implies, any line which minimizes sum of errors with respect to residues must also be orthogonal to w_1 , this needs proof, it is not too hard. I am not going to do this, but then try to argue why this is true yourself. If you are not already seeing it, this is a good place to pause and think about why this must be true. So, we won't prove this formally, but then it is not too hard to do this.

(Refer Slide Time: 08:34)



So, what are we essentially saying? So, the implication is that the conclusion is that $w_2^T w_1 = 0$, so that is what it means to say that, w_2 is orthogonal to w_1 , which means are my w_2 is actually going to be just, something this way? Not that vector, it has to be, by definition, it has to be unit length, so it would be this way, this would be w_2 , well, it could be in the opposite direction also, that is also fine. So, that would also be a valid unit vector.

But then nevertheless, that does not violate the orthogonality property at least so the orthogonality still holds for w_1 and w_2 . So, now what happens if we continue this procedure, so in 2 dimension, you have to stop here because after 2 lines, you kind of found 2 lines such that the residue of the residue is just the 0 vector.

So, because this line exactly fits the data, so there is no residue anymore. But then if your data point was in 100 dimensions, then you do not have to necessarily stop after 2 rounds you can still continue this procedure and you might get residue of the residue on so on.

(Refer Slide Time: 09:55)

By continuing this procedure, we get $\{w_1, w_2, \dots, w_d\}$ s.t. $\|w_k\|_2^2 = 1$ and $w_i^T w_j = 0 \forall i \neq j$.
 $\downarrow$
 ORTHONORMAL VECTORS

And as we keep fitting lines, what we would observe is that we would get a bunch of lines, $\{w_1, w_2, \dots, w_d\}$. So, in $\mathbb{R}^d$ by continuing this procedure we get w_1 to w_d with the property that $\|w_k\|^2 = 1$ for all k and $w_i^T w_j = 0 \forall i \neq j$, when basically you get w_3 , which is orthogonal to both w_1 and w_2 , w_4 , which is orthogonal to all the previous one and so on. So, basically, what we have ended up getting is a set of orthonormal vectors, orthonormal is just a set of vectors which are mutually perpendicular and of length 1, so, that is the orthonormal vectors. So, this is good.

(Refer Slide Time: 11:09)

Residue after round 1

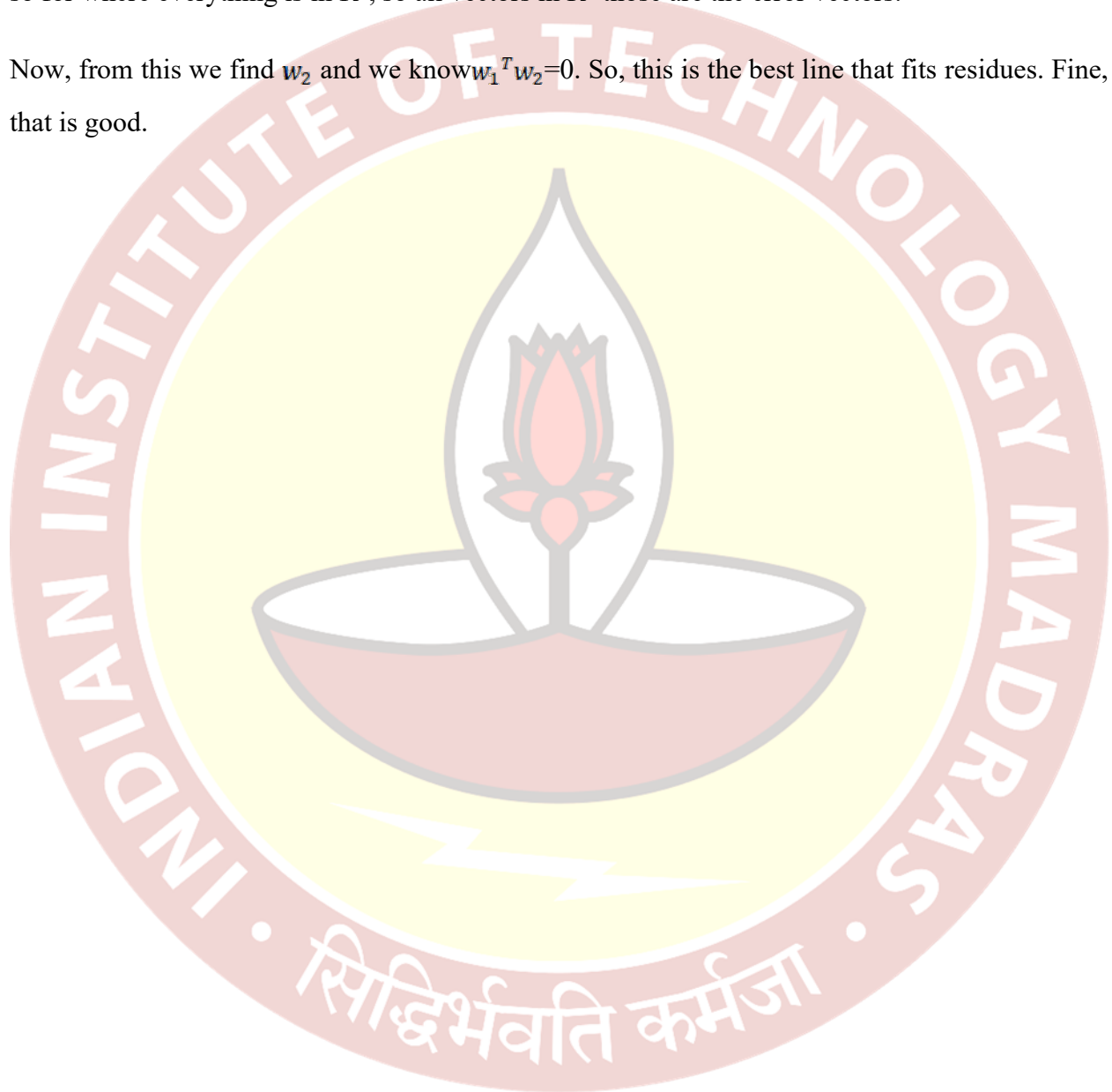
$$\{x_1 - (z_1^T w_1) w_1, \dots, x_n - (z_n^T w_1) w_1\}$$

- $w_2 \rightarrow$ Best line that fits resid
- $w_1^T w_2 = 0$

Now, just to understand what is the use of this orthonormal vectors, let us, let us understand this a little bit in a different way to explicitly computing the residue after round 1. So, we know the residue after round one is the set or the data set that is used to compute w_2 is of this form. So, $x_1 - (x_1^T w_1)w_1, \dots, x_n - (x_n^T w_1)w_1$

so for where everything is in $\mathbb{R}^d$, so all vectors in $\mathbb{R}^d$ those are the error vectors.

Now, from this we find w_2 and we know $w_1^T w_2 = 0$. So, this is the best line that fits residues. Fine, that is good.



(Refer Slide Time: 12:13)

Residues after round 2

$$\begin{aligned} & \left\{ x_1 - (x_1^T w_1) w_1 - \left((x_1 - (x_1^T w_1) w_1)^T w_2 \right) w_2 \right\} \\ &= \left\{ x_1 - (x_1^T w_1) w_1 - \left(x_1^T w_2 - \underbrace{(x_1^T w_1) w_1^T w_2}_{=0} \right) w_2 \right\} \\ &= \left\{ x_1 - (x_1^T w_1) w_1 - (x_1^T w_2) w_2 \right\} \end{aligned}$$

Now, we have two w 's. Now, let us say we go ahead and compute the residues after round 2, just to see what is happening. So, it is maybe some of you are already seeing it. But let us try to compute this explicitly. Now, what would be the residues after round 2? From round 1 data set, which was this dataset, we compute a w_2 .

And then we take the residue of each of this data point with respect to w_2 , which means that we have to take this data point subtract it out from its dot product with respect to w_2 . So, which means the first data point would be, this was the data point, which was $x_1 - (x_1^T w_1) w_1$, now, this was my original data point, I subtract out its dot product with w_2 , the same vector transpose w_2 , this is the scaling factor with respect to w_2 .

So, this is what I subtract out and then I do this for every vector, let us focus on one point to see what comes out. Now, what is this so, this is just let me try to simplify this $\{ x_1 - (x_1^T w_1) w_1 - ((x_1^T w_2) - (x_1^T w_1) w_1^T w_2) w_2, \dots \}$ which is exactly $\{ x_1 - (x_1^T w_1) w_1 - (x_1^T w_2) w_2, \dots \}$ The first term is $(x_1^T w_2) w_2$, but now, we notice that because of the fact whatever we argued earlier that w_1 and w_2 are orthogonal, this value is 0, which means that the second term is 0. So, which means this would be our first data point and so on. So, after round 2, we observed that the residue is x_1 minus x_1 's projection onto w_1 subtract out the x_1 's projection onto w_2 .

(Refer Slide Time: 14:14)

$$\begin{aligned}
 & \left\{ x_i - (x_i^T w_1) w_1 \right\} \\
 &= \left\{ x_i - (x_i^T w_1) w_1 - (x_i^T w_2) w_2 \right\} \\
 &= \left\{ x_i - (x_i^T w_1) w_1 - (x_i^T w_2) w_2, \dots \right\} \\
 & \text{Residues after d-rounds} \\
 & x_i = x_i - (x_i^T w_1) w_1 + (x_i^T w_2) w_2 + \dots + (x_i^T w_d) w_d
 \end{aligned}$$

Now, if we continue this after d rounds again this needs an argument but then it should not be too hard to see that residues after d rounds what would be the residue? It would be for all i the residue x_i would be $x_i - (x_i^T w_1) w_1 - (x_i^T w_2) w_2 \dots - (x_i^T w_d) w_d$.

Now, what do you think should be the residue after d rounds so I am in D dimension all the data points are in D dimension and I run this algorithm d times and then I look at the residue What do you think would be the residue after d rounds? In 2 dimensional case it was very obvious.

So, you had an initial set of data points you found a line, formed the projections and the error vectors with respect to the best line all the error vectors lined up so, the second line exactly fit the data set. So, the residue after round 2 would just be set of all 0 vectors because the second vector exactly fit the data points, all the vectors are lined up.

Now, the same thing would happen in D dimension if you run it long enough. So, which means if you run it for d rounds, so after d rounds, all the error vectors would simply become 0 vectors. So, because you have kind of removed all the components of this error vector, and there can only be d independent components, which are w_1 to w_d in this case.

(Refer Slide Time: 15:50)

$$= \{ \dots \}$$

Residues after d-rounds

$$x_i - (x_i^T w_1) w_1 - (x_i^T w_2) w_2 - \dots - (x_i^T w_d) w_d$$

$$x_i = (x_i^T w_1) w_1 + (x_i^T w_2) w_2 + \dots + (x_i^T w_d) w_d$$

So, which means all these vectors are actually 0 vectors after d rounds so, in $\mathbb{R}^d$. Now, that tells us that x_i itself is just $x_i - (x_i^T w_1) w_1 - (x_i^T w_2) w_2 - \dots - (x_i^T w_d) w_d$. So, basically, what we have done is again, if you know a bit of linear algebra, you can already see what is happening is that we have expressed our data set, every point in the data set in a different basis in a different orthonormal basis.

So, the $\{x_1, x_2, \dots, x_n\}$ were expressed in standard basis. Now, you are expressing the same data set in the basis of w_1 to w_d . So, basically, all we have done is doing a change of basis. So, all these data points can be expressed in any basis whatsoever. So, there is nothing, the question is what have we gained by doing this. So, we could have had the standard basis itself. But then now we are choosing to express all the data points on a different basis, which is the w_1 to w_d basis.

(Refer Slide Time: 17:07)

$x_i = (z_i^T w_1) + (z_i^T w_2) + \dots + (z_i^T w_d) + e_i$

What have we gained?

- If data lives in a "low" dimension, then residues become 0 much earlier than d rounds.

So, what, so the question is, what have we gained? What have we gained? The first thing we are gaining is the following. So, if our data lives in a low dimensional linear space or subspace, then we do not have to run d rounds. So, the residues become 0 much earlier than d rounds. So, so, we keep running this algorithm iteratively and at some point, we realized that the residues have all become 0, then there is no point in continuing further and finding a different line, so because that is not going to add anything more to our information, because all the vectors is 0.

So, what is this telling us that if it so happens that your data was in low dimensional space, then your algorithm would stop much earlier than running for d rounds.

(Refer Slide Time: 18:29)

Example: Say Dataset is such that after 3 rounds, residues become 0.

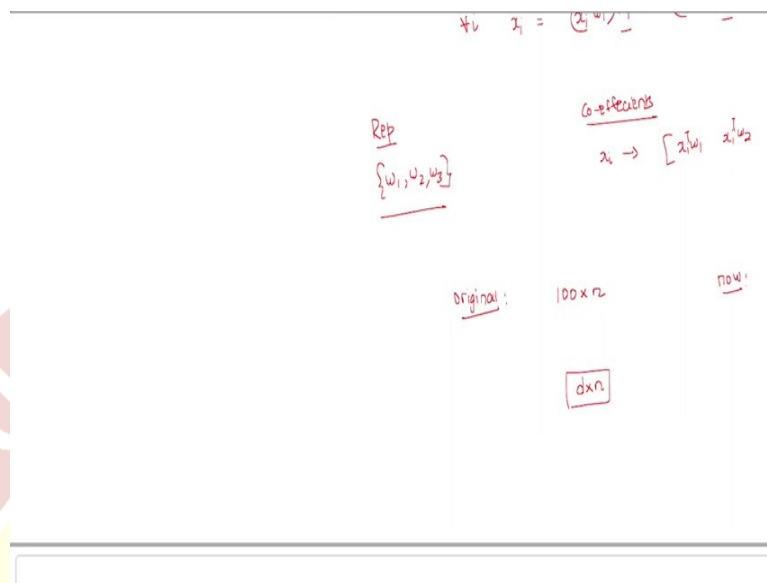
$$\text{Dataset} = \{x_1, \dots, x_n\}$$
$$x_i = (x_i^T w_1)w_1 + (x_i^T w_2)w_2 + (x_i^T w_3)w_3$$

So, why is this a useful thing now? So, this is useful, because let us take an example and see why this is an useful thing. So, for example, say data set that we have such that after 3 rounds, the residues become 0 so, you run 3 rounds, and then all the error vectors residues become 0. So, residues become 0. Now, what does that tell us?

So, the data set itself initially was in some D dimensional data set that is the usual data set that we have $\{x_1, x_2, \dots, x_n\}$ in data points all x_i are in $\mathbb{R}^d$. Now, what does it mean to say the residues become 0 after 3 rounds, it means that there exists we have found w_1, w_2, w_3 , such that for all i x_i equals $x_i - (x_i^T w_1)w_1 - (x_i^T w_2)w_2 - (x_i^T w_3)w_3$. Now, where are these w_1, w_2, w_3 ? w_1, w_2, w_3 are the lines that we find for our datasets.

So, they are all in $\mathbb{R}^{100}$, so let us say this is 100 for the moment to give some numbers. Let us see the dataset this is on 100 dimensional. So, all these three w_1, w_2, w_3 are in 100 dimension. That is good.

(Refer Slide Time: 20:12)



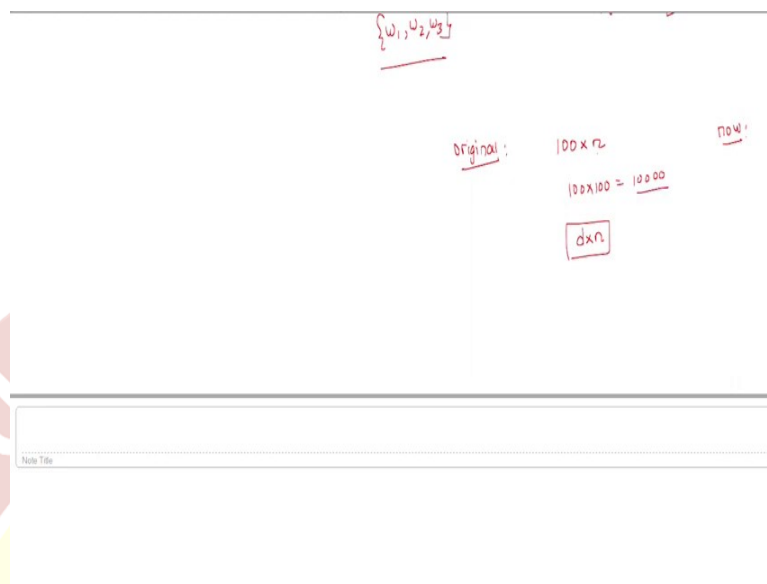
But what are we saying? We are saying that now if we have the representatives and the coefficients, so the representatives, of course, will be w_1, w_2, w_3 , which will all three lines are in 100 dimension, but the coefficients now, to reconstruct x_i , we do not need 100 numbers, so we need we need only three numbers per data point.

So, coefficient for the i th data point, so R just $x_i^T w_1, x_i^T w_2, x_i^T w_3$, it is just the dot product of the data point with respect to each of these directions that we have found so, which is in R^3 , just 3 numbers per data point. So, initially, we had 100 dimensional data points n of them.

So we had to do, the naive way to store these would be to say that we would have 100 into n numbers that that are needed to store. Now, because of the observation that after 3 rounds, the residues all become 0, then it means that for the representative, you need 3 into 100, 3 into 100 for w_1, w_2, w_3 plus, for each data point, you only need 3 points.

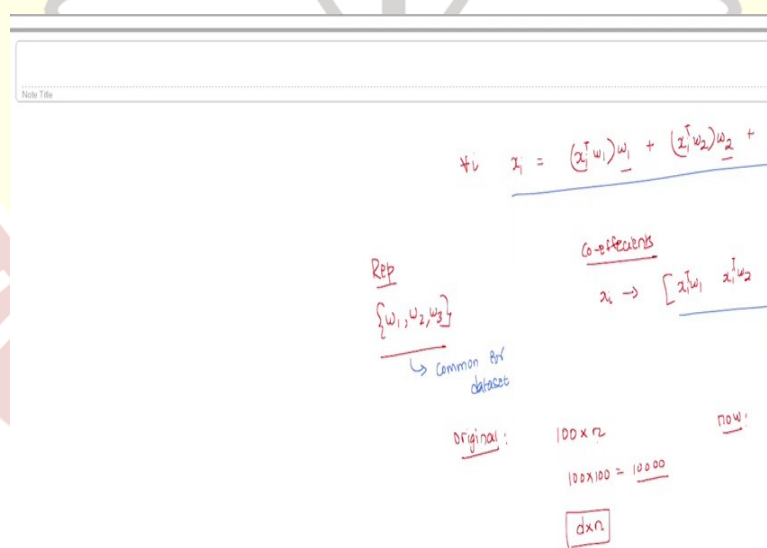
So, which would be just $3n$. So, this is, the number of this is the total number of numbers that you need to store or values that you need to store. So, in general, instead of $d \times n$, which you may have to store now you have to stored $d \times k$, where k is the the number of rounds after which the residues become 0, plus $k \times n$ x for each data point we only have k numbers instead of d numbers, so, $d \times n$ could be much, much larger than $d \times k + k \times n$, especially if k is small.

(Refer Slide Time: 22:12)



So, think of n as 100 so, then this means that this is 100×100 , that would be 10,000 whereas this is $3 \times 100 + 3 \times 100$, which is just 600. So, instead of 10,000, we have only managed to store only 600 numbers. Of course, this is assuming that you have the residues become 0 after 3 rounds.

(Refer Slide Time: 22:37)

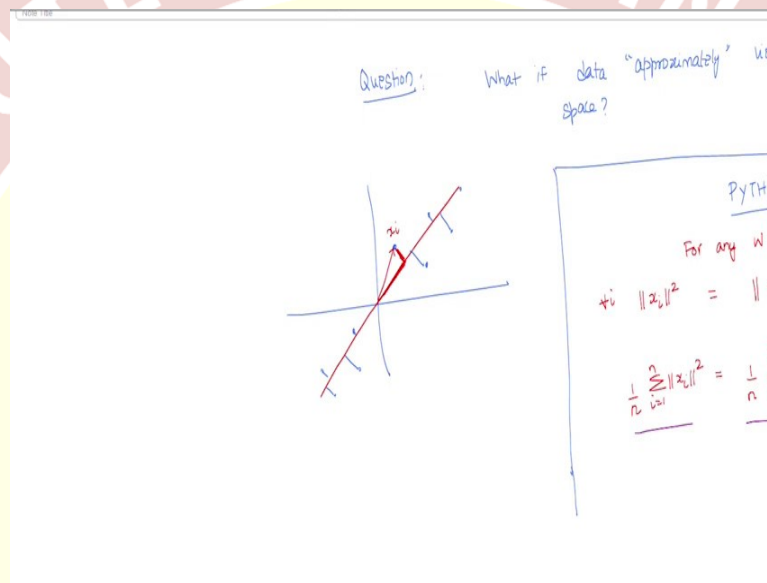


Now, what just to be very sure, so this, this is data points specific, so every data point, the numbers that we are storing, will depend on what the original data point itself, so this is data points specific. The w_1, w_2 is common for the entire data set. So, that is why they are the

representative and then this representation for this data point depends on the representative but then specific.

Now, if we want to reconstruct the data set, we could do that using the representative on coefficients by taking the linear combination of the representatives where the coefficients are specified by these, so that would give us exact reconstruction in this case. So, now, this is good.

(Refer Slide Time: 23:32)



Now, the question then is, if it so happens that after 3 rounds, the residues become exactly 0, were good, fine. But this may or may not happen in practice. And the relevant question that we should answer in practice is, what if data, approximately, lies in a low dimensional space. What does that mean?

That means that, for instance, the simplest example is the 2 dimensional space again, we might have data like this. There is no single line that fits the data so, there is no single 1 dimensional space where the data actually lies in, but still we are kind of happy with this line 1 dimensional representation, which means that we are saying we can tolerate some amount of error. So, that number has to be fixed by us how much tolerance we need, should be fixed by us.

So, how can we fix this tolerance? What should we look at to decide this tolerance? There are different things that you can look at. And the simplest way to decide on a tolerance would be to

do the following. So, here, let us look at the Pythagoras theorem. Let us take any data point. So, let us say look at any data point and look at its length.

So, maybe some data point x_i and then its length. By Pythagoras this length is the sum of the lengths of its projection onto any line not just the base line, any line plus the error vectors length so, length squared sorry length squared of x_i is the sum of the length squared of the projection onto any fixed line plus the error vector that it suffers with respect to that line.

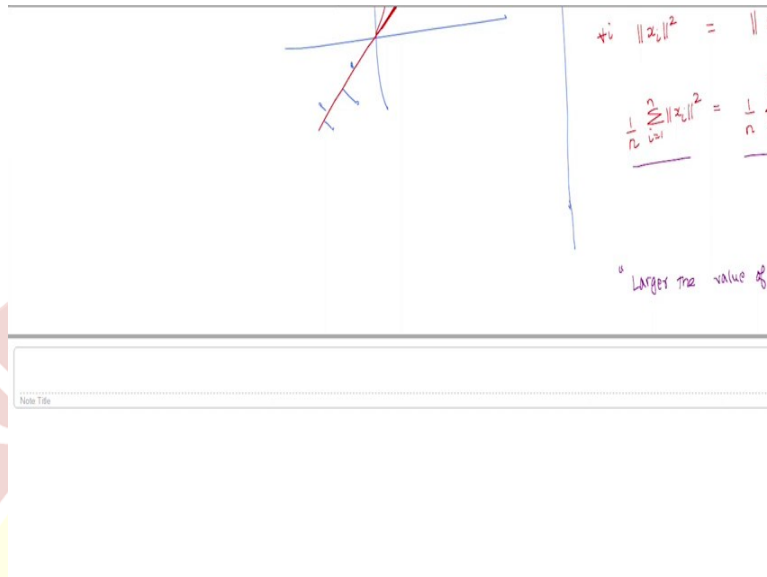
So, which means this is $\|x_i - (x_i^T w)w\|^2 + \|(x_i^T w)w\|^2$, so, this is true for any data point. Now, this is true for all i which means so, if I just average this left hand side over all i this is so for any w , so, in $\mathbb{R}^d$ this is true, where x_i is all in $\mathbb{R}^d$. If I do the average in here, I get this $\frac{1}{n} \sum_{i=1}^n \|x_i - (x_i^T w)w\|^2$

$$+ \frac{1}{n} \sum_{i=1}^n \|(x_i^T w)w\|^2.$$

For any w in $\mathbb{R}^d$, but then we typically care about w such that w 's length is 1, the fact is true for any w , if you have any w then you will have to divide this particular length of w so let us fix the length of w to be 1 so, that this is true always. So, now what are we saying we are saying that for any w the average length of the data set is equal to the average error that the data set incurs with respect to the w that we have plus this quantity.

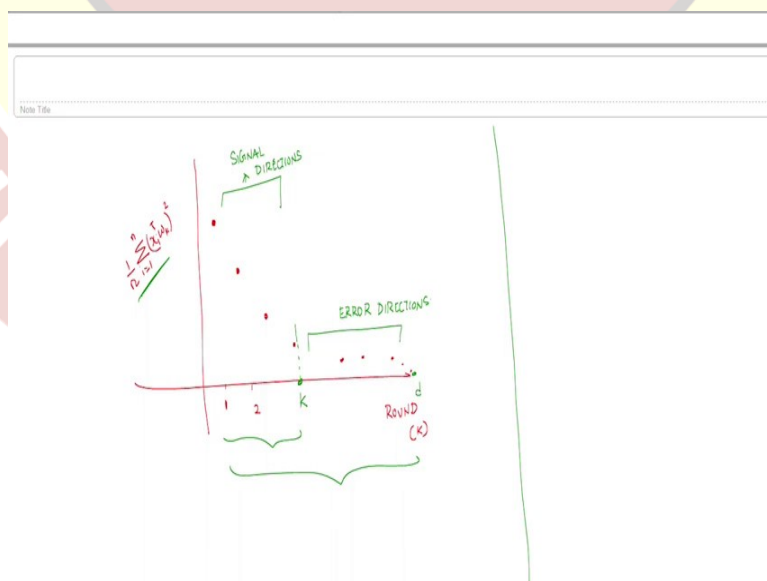
So, which means we want that w such that this quantity is as large as possible, the error should be as small as possible, or this quantity should be as large as possible which means we can look at this value after we find a w and see what happens to that value.

(Refer Slide Time: 27:20)



So, basically, what we can do is use, we will the fact that larger the value of the second term, which is $\frac{1}{n} \sum_{i=1}^n \|(x_i^T w)w\|^2$ so, this is a constant that that square of it will come out $\|w\|^2$ is anyway 1 so, (the) this is essentially $\frac{1}{n} \sum_{i=1}^n (x_i^T w)^2$, the better the fit, so this is exactly same as this, both of these are same.

(Refer Slide Time: 28:00)



So, now what we can do is do the following, so we will do the following, where we will look at rounds, as the number of rounds proceeds, let us call that round k , we will look at the average, that we will suffer with respect to w_k , the k th rounds value that we find out.

So, now, this is going to be as large as possible for w_1 , this is for round 1, because you are essentially maximizing this for round 1 and now whatever you find for round 2 is going to be lesser than so if it is greater than this, we would have found that in round 1 itself.

So, this value is going to go down, so, this is going to fall down. And let us say it falls down like this so, now if it falls down like this, there might be an elbow point, which is some k , after which these numbers are very, very small. So, what does that essentially tell us at a high level, that tells us that the top k , w 's that were found are actually the signal directions.

So, that is where the most of the information is and we will talk about what it means to say information signal and so on a little bit later but for now, we understand that, if it kind of drops down and becomes really small after point, these can be thought of as errors.

So, error directions, so, what we can do is we can kind of use a rule of thumb to say that after a particular value, if the sum of these values is divided by the total sum is greater than sum quantity threshold, let us say 95% of the total sum is kind of explained by just the top k then we will pick only the top k .

We will revisit this in a slightly different notation later that why I am not writing this time but the idea should be clear, so as the rounds progresses, we are trying to find the find the quantity and then cutting it out at some point where we think enough amount of information has been gained. Of course, that is just a rule of thumb.

So, we are not like (saying) arguing that this particular k is the best k unless we make more assumptions for both the data, we cannot really do that, but the rule of thumb would be to say that, this captures 95% of the total mass that you will have where you sum this upon all d rounds so after d rounds, this guy is going to become 0 anyway.

So, so, so now let us we have understood the algorithm, so the algorithm is kind of complete now. So, you will start with some data set, and then you can center this data set, and then start

doing this iterative procedure to get the w 's and after some point, if you feel that the error vectors, so this quantity that you are trying to maximize is too small or with respect to the total quantity that you have so far managed to achieve. Then you stopped there.

